

Metodyka pomiarów wydajności

Podsumowanie sesji projektowej — decyzje, uzasadnienia i ustalenia techniczne

Miłosz Kowalewski

29 września 2026

1. Cel sesji i wynik

Po zamknięciu etapu weryfikacji poprawności (pięć operacji polars-bio działających na Apache Ballista i LakeSail/Sail — patrz sprawozdanie_stanu_prac.md) otwartym obszarem pozostawały pomiary wydajności: ich cel, zakres i sposób wykonania. Sesja miała przekształcić ten obszar w konkretny, uzasadniony projekt.

Wynikiem jest specyfikacja docs/superpowers/specs/2026-09-29-metodyka-benchmarkow-design.md. Niniejszy dokument uzupełnia ją o **uzasadnienia decyzji, rozważone alternatywy i ustalenia techniczne zweryfikowane w kodzie w trakcie sesji** — czyli o to, czego specyfikacja, jako dokument normatywny, nie przechowuje.

Nadrzędne ograniczenie, które ukształtowało większość decyzji: **dostępna jest wyłącznie maszyna lokalna**. Chmura jest opcjonalnym rozszerzeniem skali; dostęp do niej i finansowanie pozostają do ustalenia.

2. Ustalenia techniczne zweryfikowane w trakcie sesji

2.1 Na ile obecne wykonanie jest rzeczywiście rozproszone

Ballista — logika rozproszona, dowód niepełny. Plan jest faktycznie serializowany przez własne kodery, a strona executora odtwarza operator z bajtów, budując **własną** sesję bez dostępu do pamięci klienta (ballista_genomics/src/dist_provider.rs, metoda build, komentarz przy linii 50). Shuffle po chromosomie potwierdza test z celowo rozciętymi plikami wejściowymi. Jednak klaster tworzony jest przez DFSessionContext::standalone_with_state(sta (runner.rs:162), czyli scheduler i executor żyją w **jednym procesie**. Brakujący dowód to wykonanie na osobnych procesach — wydzielone jako zadanie P0 (sekcja 4).

Plik deploy/ballista/docker-compose.yml zawiera w miejscu komend schedulera i executorów zaślepki command: ["--help"], a adnotacja w nim („zmiana dotyczy wyłącznie sposobu tworzenia sesji — kilka linii”) jest niepełna: kodery rejestrowane w konfiguracji sesji **klienta** nie przenikają do innych procesów. Scheduler i executor muszą być **własnymi binarkami** z zarejestrowanymi koderami. Ballista 53 udostępnia do tego publiczne API: SchedulerConfig::override_logical_codec / override_physical_codec + start_server (ballista-scheduler-53.0.0/src/scheduler_process.rs, config.rs:245–247) oraz ExecutorProcessConfig::override_logical_codec / override_physical_codec + start_executor_process (ballista-executor-53.0.0/src/executor_process.rs:110–170, 240). Nie wymaga to modyfikacji Ballisty.

Sail — lokalnie brak równoległości wywołań polars-bio. Tryb local-cluster to jeden proces (ustalenie z wcześniejszego etapu), a sail_pb_guard.py serializuje wszystkie wywołania pb.*() jedną blokadą threading.Lock. W efekcie lokalnie grupy chromosomów są przetwarzane przez polars-bio **sekwencyjnie**; równoległe są tylko etapy Saila wokół UDTF. Prawdziwa równoległość wywołań polars-bio wymaga workerów w osobnych

procesach, czyli — w obecnych implementacjach zarządcy workerów Saila — Kubernetesa. Wniosek dla metodyki: lokalne pomiary Saila systematycznie zaniżają jego potencjał i muszą być opisane jako **asymetria**.

2.2 Sufit równoległości i nierównomierność obciążenia

Oba wzorce rozpraszania oparte na kluczu chrom mają sufit równoległości równy liczbie chromosomów (≈ 24 dla człowieka), a obciążenie jest nierówne: chr1 (≈ 249 Mb) jest ok. 5,3 raza dłuższy od chr21 (≈ 47 Mb). Przy wielu węzłach czas wyznacza największa partycja. Przy maksymalnie 3 emulowanych węzłach lokalnie sufit nie jest osiągalny — zjawisko staje się istotne dopiero w skali chmurowej. Drobniejsze partycjonowanie (chromosom + przedziały genomu, z obsługą interwałów na granicach) odłożono jako kierunek dalszych prac.

2.3 Budowa indeksu w złączeniu interwałowym

BioPhysicalPlanner tworzy IntervalJoinExec w trybie PartitionMode::CollectLeft (vendor/datafusion-bio-function-ranges/src/physical_planner/bio_physical_planner.rs:145). W tym trybie operator wymaga na lewym wejściu Distribution::SinglePartition (joins/interval_join.rs:396–399), a indeks lewej strony budowany jest przez self.left_fut.once(...) (interval_join.rs:466) — **raz na instancję planu**. Osobne procesy nie współdzielą instancji planu, więc po rozproszeniu indeks budowany jest **co najmniej raz na executor**. Konsekwencja: w wersji rozproszonej koszt budowy indeksu ma większą wagę niż lokalnie, co uzasadnia hipotezę P3 (zmiana rankingu algorytmów). Dla porównania tryb Partitioned buduje indeks per partycja z hash-partycjonowanych danych (interval_join.rs:481) — na mniejszych podzbiorach.

2.4 Limit wiadomości gRPC przy broadcast

Dotychczas przyjmowano limit 16 MB jako twardy sufit wzorca broadcast (nearest, coverage). Kod Ballisty 53 pokazuje, że limit jest **konfigurowalny bez modyfikacji silnika** w trzech miejscach:

Miejsce	Ustawienie	Domyślnie
klient	ballista.client.grpc_max_message_size (konfiguracja sesji; extension.rs:442)	16 MiB
scheduler	SchedulerConfig:: grpc_server_max_decoding_message_size (config.rs:382)	16 MiB
executor	ExecutorProcessConfig:: grpc_max_decoding_message_size, grpc_max_encoding_message_size (executor_process.rs:144–146)	16 MiB

W trybie standalone scheduler dekoduje z limitem klienta (standalone.rs:102). Szacunek rozmiaru: interwał w Arrow IPC to ok. 25–30 B, więc 16 MiB mieści ok. 0,5–0,7 mln interwałów — fBrain (199 tys.) się mieści, exons (439 tys.) jest na granicy, ex-anno (1,19 mln) i chainRn4 (2,35 mln) nie. Skuteczność podniesienia limitu zweryfikuje przebieg smoke.

Istotny koszt wzorca: tabela broadcastowana podróżuje **wewnątrz planu**, a plan trafia do każdego zadania — przesyłany wolumen to w przybliżeniu ładunek $\times$ liczba zadań (dla ex-anno rząd $35 \text{ MB} \times 6 \approx 200 \text{ MB}$ na zapytanie). Jest to mierzalna właściwość wzorca, raportowana jako broadcast_bytes. Plan zapasowy: przekazywanie w ładunku ścieżki do pliku Parquet zamiast danych.

2.5 Algorytmy w polars-bio 0.28

Parametr `algorithm` istnieje wyłącznie w `overlap` (`polars_bio/range_op.py:113, 137`), wartości: `Coitrees` (domyślny), `IntervalTree`, `ArrayIntervalTree`, `Lapper`, `SuperIntervals`. Na stronie wyników wydajności `polars-bio` porównywane są biblioteki (`bioframe`, `pyranges`, `GenomicRanges` i in.), nie algorytmy — założenie „`Coitrees` jest najszybszy” jest domyślnym wyborem, nie opublikowanym wynikiem, co zwiększa wartość P3.

2.6 Metodyka polars-bio-bench

Repozytorium `biodatageeks/polars-bio-bench` (punkt odniesienia dla porównywalności):

- orkiestracja: `src/run-benchmarks.py` + pliki YAML w `conf/` (`benchmark_small.yaml`, `common.yaml`); klucze `m.in.` `num_repeats`, `num_executions`, `threads`, `parallel`, `repartition`, `polars_bio_coordinate_system`, `polars_bio_consume_mode`;
- czas: `timeit.repeat(invocation, repeat=num_repeats, number=num_executions)`, zapisywane `min / max / średnia`; brak rozgrzewki; w `benchmark_small.yaml` 3 powtórzenia $\times$ 1 wykonanie, wątki 1, 2, 4;
- wczytanie danych **poza** pomiarem (dane ładowane i buforowane przed wywołaniem);
- wszystkie narzędzia w **jednym procesie** Pythona;
- pamięć: osobne przebiegi przez `mprof` (`memory_profiler`, `src/run-memory-profiler.py`);
- dane: 9 zbiorów `AList` (`databio`, `databio-8p` — `Parquet`, wariant 8-plikowy), zbiory syntetyczne, pary identyfikowane jako `a-b`; klasy wyników `S / M / L / XL`;
- wyniki: CSV.

Przejęto: format konfiguracji, dane i pary, klasy rozmiaru, statystyki `min/średnia/max`. Świadome odstępstwa i ich uzasadnienie — sekcja 3, decyzja D9.

2.7 Środowisko lokalne

Parametr	Wartość
CPU	AMD Ryzen 5 4600H, 6 rdzeni / 12 wątków; rdzeń k = wątki $\{2k, 2k+1\}$ (<code>lscpu -e</code>)
RAM hosta	7,4 GB (prawdopodobnie 8 GB fizycznie, część dla zintegrowanej grafiki)
RAM WSL cgroup	3,5 GB (domyślna połowa — brak pliku <code>.wslconfig</code>), swap 1 GB v2 z kontrolerem <code>memory</code> , bez <code>systemd</code> (limity wymagałyby ręcznej konfiguracji z <code>sudo</code>)
Docker	Docker Desktop po stronie Windows (dzieli pamięć maszyny wirtualnej WSL)
Dysk	176 GB wolne

Zużycie pamięci przy otwartym tylko VS Code (pomiar w trakcie sesji):

Składnik	Pamięć
VS Code — procesy Windows	$\approx$ 815 MB
VS Code — procesy wewnątrz WSL (Pylance $\approx$ 310, host rozszerzeń $\approx$ 250, Claude Code $\approx$ 280, serwer $\approx$ 105)	$\approx$ 950 MB
aplikacje w tle (Bitwarden, SteelSeries GG, SearchApp, OneDrive, NVIDIA)	$\approx$ 600 MB
usługi systemowe, Defender	$\approx$ 1 GB

Środki: pomiary bez VS Code (uruchamiane z Windows Terminal), zamknięcie aplikacji w tle, `.wslconfig` z `memory=5GB`, `swap=4GB` — łącznie ok. dwukrotny wzrost pamięci dostępnej dla eksperymentów ($\approx$ 4,5 GB zamiast $\approx$ 2,3 GB). Każdy przebieg ze swapem jest nieważny.

2.8 Semantyka operacji (sonda na polars-bio 0.28)

Docstring coverage nie precyzuje, która tabela otrzymuje wynik, dlatego semantykę ustalono empirycznie: wszystkie pięć operacji wykonano w polars-bio 0.28 na przykładzie $A = \{[5,20), [15,30), [40,55), [70,80)\}$, $B = \{[10,25), [45,50), [52,65), [88,95)\}$ (chr1, 0-based, półotwarte). Ilustracje w wersji przystępnej są generowane z tych samych wywołań (raporty/img/generuj_rysunki.py).

Operacja	Wynik	Kolumny
overlap(A, B)	pary nakładające się: ([5,20],[10,25)), ([15,30],[10,25)), ([40,55],[45,50)), ([40,55],[52,65))	chrom_1, start_1, end_1, chrom_2, start_2, end_2
nearest(A, B)	dla każdego interwału A jeden najbliższy z B ($k = 1$, nakładające się mają odległość 0): [5,20)→[10,25) 0; [15,30)→[10,25) 0; [40,55)→[45,50) 0; [70,80)→[52,65) 5	jak overlap + distance
coverage(A, B)	wiersze A z liczbą par zasad pokrytych przez B: 10, 10, 8, 0	chrom, start, end, coverage
merge(A)	[5,30) (2 interwały), [40,55) (1), [70,80) (1)	chrom, start, end, n_intervals
subtract(A, B)	fragmenty A po usunięciu części pokrytych przez B, per interwał A : [5,10), [25,30), [40,45), [50,52), [70,80)	chrom, start, end

Dwie obserwacje istotne dla metodyki:

- **remisy w nearest:** dla [40,55) oba interwały B nakładają się (odległość 0), a polars-bio zwraca jeden z nich. Wybór sąsiada przy remisie nie jest częścią kontraktu operacji — stąd suma kontrolna dla nearest obejmuje odległość, a nie identyfikację sąsiada (zgodnie z metodyką istniejących testów);
- **odległość** to liczba pozycji przerwy: dla [70,80) i [52,65) wynosi $70 - 65 = 5$.

3. Decyzje

#	Obszar	Rozważane warianty	Decyzja	Uzasadnienie
D1	Pytania badawcze	Ballista vs Sail jako oś główna; opłacalność; skalowalność; algorytmy	P1 opłacalność, P2 skalowalność jako główne; P3 algorytmy poboczne; Ballista vs Sail przekrojowo; P0 „czy rozproszone” jako warunek wstępny	porównanie silników wynika samo z porównania każdego z nich do wspólnego punktu odniesienia; P0 zamykają pomiary na klastrze wieloprocesowym i osobne zadanie
D2	Skalowanie	silne i słabe	tylko silne	słabe wymaga generatora danych; przyjęto wyłącznie dane rzeczywiste

#	Obszar	Rozważane warianty	Decyzja	Uzasadnienie
D3	Punkt odniesienia	A: węzeł tej samej wielkości; B: maszyna = cały klaster	oba ; wybór głównego do ustalenia	lokalnie oba są darmowe; B odpowiada na zarzut z metryki COST
D4	Rola pomiarów lokalnych	pełnoprawne wyniki; tylko walidacja narzędzi	pełnoprawne wyniki	praca musi się obronić bez chmury
D5	Treść danych	zbiory polars-bio-bench; generator; oba; dane kohortowe VCF	zbiory AIList z polars-bio-bench	porównywalność z publikacją polars-bio; rzeczywiste rozkłady; gotowa gradacja rozmiarów; VCF lokalnie niewykonalne
D6	Sposób składowania	Parquet teraz, lakehouse później; składowanie jako zmienna; tylko lakehouse	Parquet teraz, lakehouse później (wymienne źródło danych)	architektura lakehouse nie jest jeszcze sprecyzowana; minimalne ryzyko przeróbek; CSV odpada niezależnie (parsowanie tekstu zdominowałoby czasy)
D7	Metryki	czas; pamięć; shuffle; fazy	wszystkie cztery (fazy i shuffle tam, gdzie silnik je udostępnia)	czas i pamięć odpowiadają na P1-P2, shuffle i fazy wyjaśniają „dlaczego”
D8	Sail lokalnie	asymetria + próba K8s; sama asymetria; obejście blokady pulą procesów	asymetria + ograniczona czasowo próba K8s	obejście blokady nie jest rozproszeniem Saila (metodycznie wątpliwe); próba K8s może dać symetrię tanim kosztem
D9	Narzędzie	A: własny orkiestrator, osobne procesy; B: pytest-benchmark; C: hyperfine	A	B mierzy w jednym procesie (brak pamięci executorów, przeciekający kontekst polars-bio); C mierzy czas całego procesu z importem polars-bio (sekundy) i nie zbiera pamięci, faz ani shuffle
D10	Odstępstwa — od polars-bio-bench		pomiar od pliku do wyniku; osobne procesy ; pamięć przez /proc w tym samym przebiegu; 5 powtórzeń + rozgrzewka; mediana	równoległe czytanie danych jest częścią korzyści z rozproszenia; izolacja stanu; brak narzutu próbkowania; odporność na zakłócenia
D11	Zakres P3	algorytmny w overlap; shuffle vs broadcast; drobniejsze partycjonowanie; partycjonowanie w lakehouse	algorytmny w overlap	tanie (parametr istnieje); porównanie wzorców wynika z P1 bez osobnego eksperymentu; partycjonowanie lokalnie niewidoczne i kosztowne

#	Obszar	Rozważane warianty	Decyzja	Uzasadnienie
D12	P0	z pomiarem czasu; bez	bez pomiarów czasu — wyłącznie dowód rozproszenia	P0 pyta „czy”, nie „jak szybko”; czasy na pięciu interwałach nie mają znaczenia

4. Zadanie wstępne P0 — Ballista w osobnych procesach

Zaakceptowany projekt, wykonywany przed narzędziem pomiarowym:

- binarki scheduler i executor z zarejestrowanymi koderami (API z sekcji 2.1), executor z flagą `--no-codecs`;
- tryb `remote_with_state()` w `runner.rs`, gdy podany jest adres schedulera;
- uruchomienie natywne (bez Dockera: Docker Desktop dzieli pamięć maszyny wirtualnej WSL, a budowanie obrazu oznacza kompilację Rusta od zera zamiast przyrostowej);
- 2 executory po 2 sloty zadań — ograniczenie slotów wymusza rozłożenie 4 partycji na oba executory;
- test `tests/test_ballista_multiprocess.py` z czterema dowodami: (1) cztery różne PID-y,
(2) wynik zgodny z `polars-bio`, (3) zadania wykonane na obu executorach i shuffle między nimi, (4) kontrola negatywna: executor bez koderów $\Rightarrow$ błąd zapytania.

Osobne procesy dowodzą, że plan przekracza granicę procesu, a executor liczy bez dostępu do pamięci klienta; dowody (3) i (4) wykluczają odpowiednio „wszystko na jednym executorze” i „dekodowanie gdzie indziej niż w executorze”. Uruchomienie na kilku maszynach nie wnosi jakościowo nowego dowodu — dochodzi jedynie fizyczna sieć. Pozostaje znane ryzyko poza zakresem P0: ładunek planu przenosi ścieżki plików, więc procesy muszą współdzielić system plików.

5. Metodyka w skrócie

Element	Ustalenie
Węzeł	1 rdzeń fizyczny = 2 wątki; $N \in \{1, 2, 3\}$; rdzeń 0 — system, scheduler, klienci
Warianty	<code>polars-bio A</code> , <code>polars-bio B</code> , Ballista (N executorów), Sail (1 proces na N rdzeniach)
Dane	<code>datbio-8p</code> ; 16 kandydackich scenariuszy lokalnych; reguła kalibracji: szczyt pamięci $\leq \frac{1}{3} \text{MemAvailable}$
Pomiar	od wysłania zapytania do ostatniego wiersza; 1 rozgrzewka + 5 rund w losowej kolejności; przebieg kontrolny na początku i końcu bloku (próg 10%)
Poprawność	każdy przebieg porównany z <code>polars-bio A</code> (liczba wierszy + suma kontrolna niezależna od kolejności)
Nieważność	swap, niezgodność wyniku, timeout 20 min
Wolumen	≈ 860 przebiegów ($P1 + P2$) + 48 EXPLAIN ANALYZE + ≈ 360 (P3); rząd kilku godzin pracy maszyny
Kolejność	<code>smoke</code> $\rightarrow$ kalibracja $\rightarrow P1$ ($N = 1, 3$) $\rightarrow P2$ ($N = 2$) $\rightarrow P3 \rightarrow$ analiza

Pełna macierz, schemat wyników, protokół runnera i definicja sumy kontrolnej — w specyfikacji.

6. Plan działania

A. Dokumentacja. Specyfikacja i dwa podsumowania (ten dokument i wersja przystępna); przegląd; commit.

B. Plan implementacji. Plan zadań z testami (docs/superpowers/plans/); wybór trybu wykonania — rekomendowane wykonanie w jednej sesji (mniejsze zużycie tokenów niż osobni subagenci per zadanie).

C. Implementacja (TDD, commit po każdym zadaniu, stan w checkboxach planu — odporne na przerwanie sesji):

1. P0 — Ballista w osobnych procesach;
2. dane: pobranie databio-8p, CSV → Parquet;
3. narzędzie: metryki, suma kontrolna (Python + Rust), runnery, orkiestrator, YAML, smoke;
4. instrumentacja Saila (czas pb.*(), czas blokady), parametr algorithm, limity gRPC;
5. próba Saila na Kubernetesie (wymaga wcześniejszej zmiany .wslconfig — restart WSL).

D. Pomiary (uruchamiane z terminala po liście kontrolnej pamięci): kalibracja (+ oszacowanie kosztów chmury) → P1 → P2 → P3 → analiza.

7. Kwestie do ustalenia na dalszym etapie

- główny wariant punktu odniesienia (A czy B);
- dostęp do chmury i finansowanie;
- architektura lakehouse i sposób podłączenia źródła danych;
- zakres operacji complement i cluster.

8. Literatura dodana w trakcie sesji

McSherry F., Isard M., Murray D.G. *Scalability! But at what COST?* 15th Workshop on Hot Topics in Operating Systems (HotOS XV), USENIX, 2015 — dopisana do analiza_literatury.md jako pozycja [10]; uzasadnia porównywanie wariantów rozproszonych z silnym punktem odniesienia na jednej maszynie.